

Gün 11 — FastAPI Proje İskeleti ve Pydantic Modelleri

Hedef

Backend servisinin temel iskeletini kurmak: çalışan bir FastAPI uygulaması, API yanıt şeması için bir Pydantic model dosyası ve `uvicorn` ile ayağa kalkan, sağlık durumu (health-check) döndüren bir servis.

Yaklaşım

1. Paket yapısı

`backend/app/` bir Python paketi haline getirildi (`__init__.py` eklendi) ki `uvicorn` `app.main:app` komutu `backend/` dizininden çalıştırılabilsin. Bu yapı, Gün 13'te `scripts/` altındaki analiz modüllerinin (`detect_trailer`, `entropy`, `size_analysis`, `extract`, `video_metadata`, `analyze`) `backend/app/pipeline.py` içinden import edilmesiyle uyumlu olacak şekilde düşünüldü.

2. `backend/app/main.py` — FastAPI uygulaması

`FastAPI(title=..., description=..., version="0.1.0")` ile uygulama nesnesi oluşturuldu. Plan'da health-check endpoint'inin yolu belirtilmediğinden en sade seçenek olan kök yol (`GET /`) kullanıldı; yanıt tipi `response_model=HealthResponse` ile Pydantic modeline bağlandı.

3. `backend/app/models.py` — Pydantic modelleri

İki model tanımlandı: - `HealthResponse` — yalnızca `status: str = "ok"` alanı, Gün 11'in kabul kriterini karşılamak için. - `AnalyzeResponse` — Gün 14'te tam olarak işlenecek nihai API yanıt şeması için şimdiden iskelet olarak açıldı: `polyglot_status: bool` (zorunlu, zaten Gün 1-10'da netleşmiş ana sinyal), `threat_score`, `extracted_video_url`, `analysis_summary` alanları `Optional/None` varsayılanlı bırakıldı — böylece model Gün 11'de import edilip kullanılabilir durumda, ama hesaplama mantığı (ağırlıklı skor, dinamik özet metni) bilinçli olarak Gün 14'e bırakıldı (Gün 10 raporunda alınan "her sinyal kararı kendi gününde verilsin" disiplini sürdürüldü).

4. `backend/requirements.txt`

Gün 1'de zaten oluşturulmuş dosya (`fastapi==0.141.1`, `uvicorn==0.52.3`, `python-multipart==0.0.32`, `opencv-python==5.0.0.93`, `Pillow==12.3.0`, `numpy==2.4.6`, `matplotlib==3.11.1`) Gün 11'in istediği tüm paketleri zaten içeriyordu; ek bir değişikliğe gerek kalmadı, yalnızca doğrulandı.

Dosyalar

```
backend/app/__init__.py    # paket işareti
backend/app/main.py        # FastAPI app + health-check endpoint
backend/app/models.py      # HealthResponse, AnalyzeResponse
```

Test Sonuçları

Proje kökündeki `.venv` (Gün 1'de kurulmuş) aktive edilip `backend/` dizininden servis başlatıldı:

```
$ cd backend && source ../.venv/bin/activate
$ uvicorn app.main:app --port 8000
```

İstek	Sonuç
GET /	<code>{"status": "ok"}</code>
GET /docs	HTTP 200 (Swagger UI erişilebilir)
GET /openapi.json → info	<code>{'title': 'Polyglot / Steganaliz Servisi', 'version': '0.1.0', ...}</code>

```
$ curl -s http://127.0.0.1:8000/
{"status": "ok"}
$ curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/docs
200
```

Kabul Kriterleri — Durum

- [x] `uvicorn` ile başlatılan servis `http://127.0.0.1:8000` üzerinde `{"status": "ok"}` döndüren bir health-check endpoint'i sunuyor

Notlar / Riskler

- Yok. README.md'deki kurulum bölümü `backend/` içinde ayrı bir `.venv` öneriyor; pratikte proje kökündeki tek `.venv` (Gün 1'den beri tüm `scripts/` modülleri için kullanılan) backend için de kullanıldı — bağımlılıklar zaten ortak (`requirements.txt` ile birebir aynı paketler). README, Gün 19'da (dokümantasyon günü) bu gerçek kurulumla göre güncellenecek.